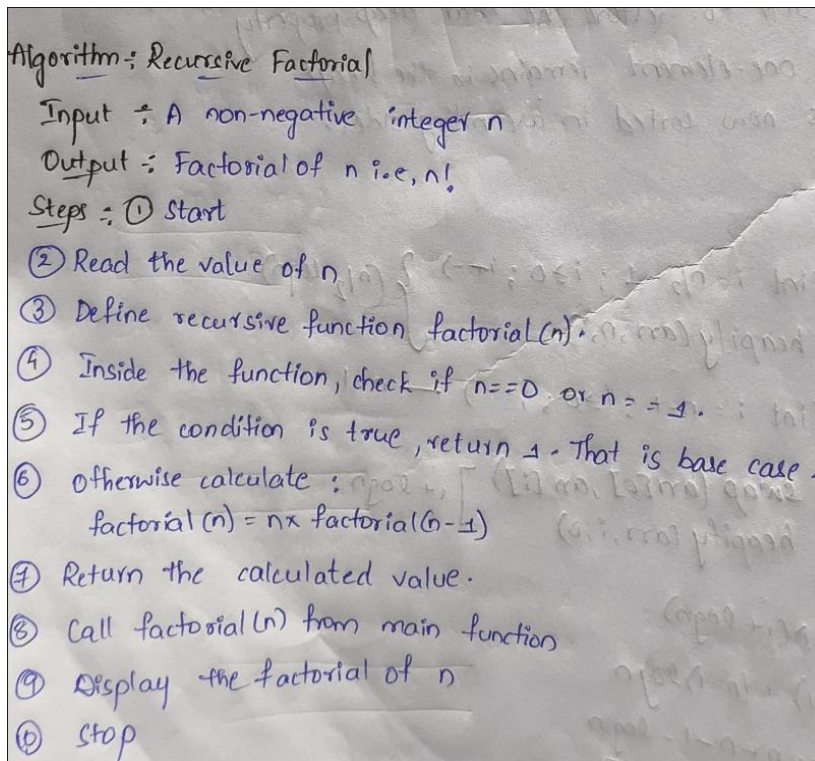


Practical - 4

Aim: To implement and time analysis of factorial program using iterative and recursive method.

1) Finding factorial using recursive method

Algorithm:



Algorithm: Recursive Factorial
Input : A non-negative integer n
Output : Factorial of n i.e., $n!$
Steps : ① Start
② Read the value of n .
③ Define recursive function $\text{factorial}(n)$.
④ Inside the function, check if $n == 0$ or $n == 1$.
⑤ If the condition is true, return 1. That is base case.
⑥ otherwise calculate : $\text{factorial}(n) = n \times \text{factorial}(n-1)$
⑦ Return the calculated value.
⑧ Call $\text{factorial}(n)$ from main function.
⑨ Display the factorial of n .
⑩ stop

Program:

```
#include <iostream>
using namespace std;

class Factorial
{
public:
    int factr(int n)
    {
        if (n == 0 || n == 1)
        {
            return 1;
        }
        else
        {
            int result = n * factr(n - 1);
            return result;
        }
    }
}
```

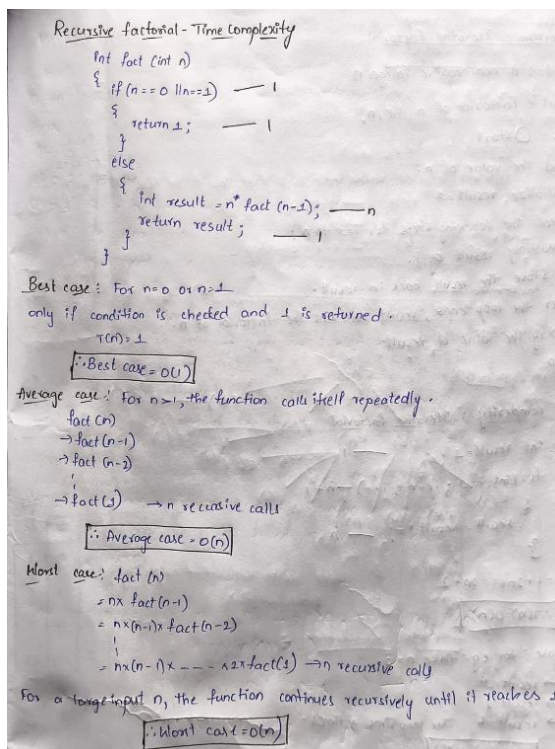
Name : Vemalatha Bhimireddy
Enrollment Number: 92460118162

```
    }  
};  
  
int main()  
{  
    int n, result;  
    Factorial obj;  
  
    cout << "Enter the value to find its factorial: ";  
    cin >> n;  
  
    result = obj.factr(n);  
  
    cout << "Factorial of " << n << " is " << result << endl;  
  
    return 0;  
}
```

OUTPUT :

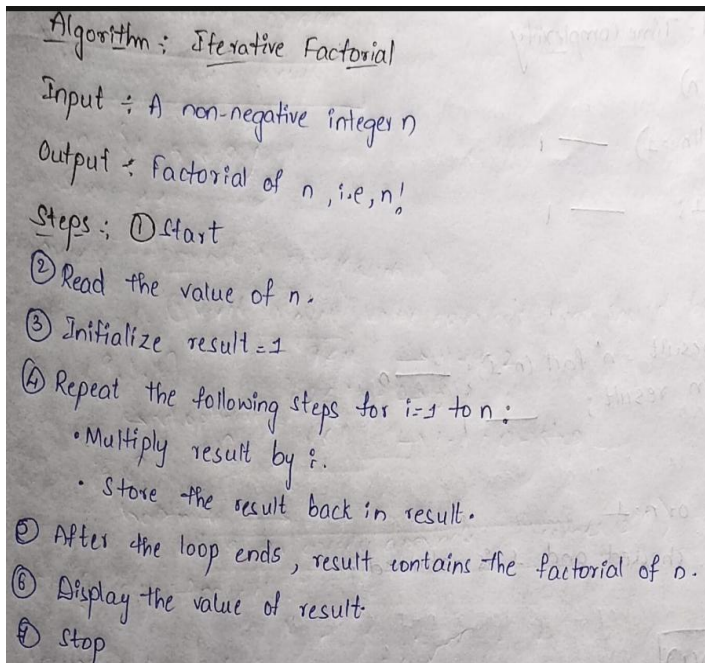
```
compilation terminated.  
PS C:\Users\vemal\OneDrive\Documents\DSA> g++ recursive_factorial.cpp -o recursive_factorial  
PS C:\Users\vemal\OneDrive\Documents\DSA> .\recursive_factorial.exe  
Enter the value to find its factorial: 5  
Factorial of 5 is 120
```

Time Complexity :



2.) Finding factorial using iterative method

Algorithm :



Program :

```
#include <iostream>
using namespace std;

class Factorial
{
public:
    int fact(int n)
    {
        int result = 1;

        for (int i = 1; i <= n; i++)
        {
            result = result * i;
        }

        return result;
    }
};

int main()
{
    int n, result;
    Factorial obj;
    Name : Vemalatha Bhimireddy
    Enrollment Number: 92460118162
```

```
cout << "Enter the value to find its factorial: ";  
cin >> n;  
  
result = obj.fact(n);  
  
cout << "Factorial of " << n << " is " << result << endl;  
  
return 0;  
}
```

Output :

```
PS C:\Users\vemal\OneDrive\Documents\DSA> g++ iterative_method.cpp -o iterative_method  
PS C:\Users\vemal\OneDrive\Documents\DSA> .\iterative_method.exe  
Enter the value to find its factorial: 6  
Factorial of 6 is 720
```

Time Complexity :

Time complexity : Iterative factorial

```
int result = 1;  
for (int i = 1; i <= n; i++)  
{  
    result = result * i;  
}
```

$T(n) = 1 + 2n + 1 = 2n + 2$
 $\therefore T(n) = O(n)$

Best case : for eg: take $n = 3$
 $i = 1 \rightarrow \text{result} = 1$
 $i = 2 \rightarrow \text{result} = 2$
 $i = 3 \rightarrow \text{result} = 6$ The loop runs 3 times.
 $\therefore \text{Best case} = O(n)$

Average case : for eg: take $n = 5$
 $i = 1 \rightarrow \text{result} = 1$
 $i = 2 \rightarrow \text{result} = 2$
 $i = 3 \rightarrow \text{result} = 6$
 $i = 4 \rightarrow \text{result} = 24$
 $i = 5 \rightarrow \text{result} = 120$
The loop runs 5 times.
 $\therefore \text{Average case} = O(n)$

Worst case : Similarly in worst case, for value n , the loop runs n times.
 $\therefore \text{Worst case} = O(n)$



Marwadi
University
Marwadi Chandarana Group



Faculty of Engineering & Technology
Department of CSE-AI&ML
Design and analysis of algorithm – 01AI0506

Conclusion: The factorial program was successfully implemented using both iterative and recursive methods. The time analysis showed that both methods have linear time complexity, but the iterative method is generally more efficient as it avoids the overhead of recursive function calls.